🎲 Sample, Then Diagonalize

Lab 1 of 3 · building the SQD method on a small model system

James Weaver · Qiskit Fall Fest 2026

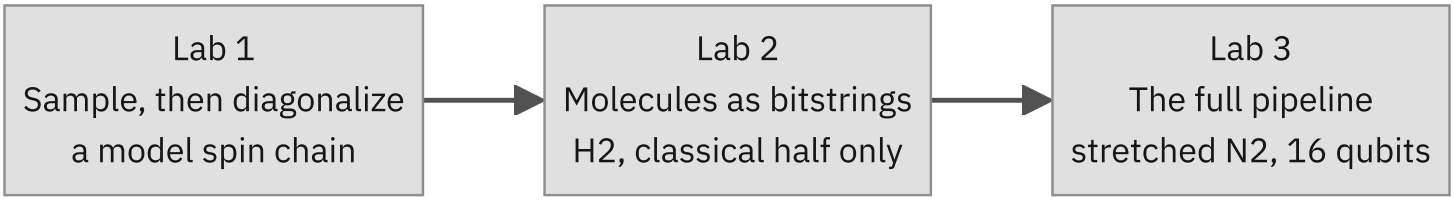
Background

In **May 2016**, IBM put a 5-qubit quantum processor on the cloud, the first one anybody could use from a browser.

A decade later, this series covers **sample-based quantum diagonalization (SQD)**, the method behind several of the largest quantum chemistry calculations run on quantum hardware so far.

> The idea behind SQD: a quantum processor does not have to produce the answer to your problem. It only has to tell you where in the space of possibilities the answer lives.

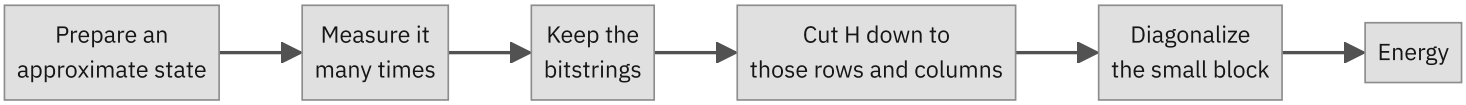
Where this lab fits



This session is **Lab 1**, which builds the whole method on a small model system instead of a molecule, so you can look at every piece directly.

What you need first: qubits, circuits, gates, and measurement in Qiskit. No chemistry is needed or assumed. That starts in Lab 2.

How the method works



The quantum processor suggests which basis states matter. A classical solver then works out the right combination of them. That split is the whole idea.

> The quantum computer proposes; the classical computer disposes.

Three terms you will need

TERM	WHAT IT MEANS
Hamiltonian	The energy rulebook. Written as a matrix: a square grid of numbers with one row and one column per possible configuration.
Ground state	The lowest-energy state the rulebook allows. It might be one configuration, or a mix of many.
Diagonalizing	The standard procedure that pulls the list of allowed energies out of that matrix. In code it is one call.

The numbers in that list are called **eigenvalues**, and the smallest one is the ground-state energy we are after.

Ground-state energies matter well beyond physics. They help determine whether a drug molecule binds its target, how a battery electrode stores charge, and which way a magnet points.

> The notebook builds all three ideas up properly. For now, "energy rulebook", "lowest-energy state", and "read off the list" will get you through.

The model system

A row of small magnets, one per qubit. Neighboring magnets prefer to point the same way, and a sideways field of strength g mixes together arrangements that differ by a single flip:

$$H = -\sum_i Z_i Z_{i+1} \; - \; g \sum_i X_i$$

The big sigma is just a loop: repeat the thing after it for each position and add up the results.

- The first term **scores** each arrangement. Neighbors that agree lower the energy.
- The second term **connects** arrangements that are one flip apart. It is the sideways field.

That second term is what makes the problem quantum. Turn it off and the ground state is a single bitstring. Turn it on and the ground state becomes a blend of several.

> With three magnets there are only 8 arrangements, so the whole rulebook fits on one screen as an 8-by-8 table, with g on a slider you can drag.

Why we cannot just brute-force it

Each magnet you add **doubles** the size of the table. For n qubits the Hamiltonian is a $2^n \times 2^n$ matrix.

QUBITS	MEMORY TO STORE ONE STATE
8	negligible
30	~17 GB
50	**~18 petabytes**

That is the cost of just *writing down* a single candidate state. Diagonalizing it is far worse.

> So exact diagonalization runs out of room long before it reaches interesting problems. Fifty qubits is a small molecule.

Why a short list of bitstrings is enough

Any state of 8 qubits is a blend of all 256 bitstrings, and each one comes with a number called its **amplitude**. Square that number and you get the probability of measuring that bitstring.

In the worst case every amplitude matters and no shortlist exists. But in many physical systems almost all of the weight sits on **a small number of bitstrings**.

The ground state of our magnet chain is a good example: two arrangements dominate, a few one- and two-flip neighbors contribute a little, and the remaining 200-plus are close to zero.

When that happens you do not need the full matrix. You can **crop** it: keep only the rows and columns for the bitstrings that matter, and diagonalize the small block that is left.

> Concentration is a property of the particular problem, not a rule of nature. It is what SQD takes advantage of, and where it is absent, the method struggles.

Where the shortlist comes from

Measure a state and you get one bitstring back, with a probability equal to its amplitude squared. Heavy bitstrings come back often. Negligible ones almost never do, which is fine, because those are the ones we wanted to drop anyway.

So measurement gives you a way to find the important basis states without knowing the answer in advance.

Two properties make the cropped calculation trustworthy:

- **It is variational.** The cropped answer can never come out below the true ground energy, so you always know which direction your error points.
- **It only improves.** Adding a bitstring either lowers the answer or leaves it alone. It never makes it worse.

The main result

The notebook builds a deliberately rough circuit on purpose: a GHZ state, which is a superposition of all-zeros and all-ones, plus a small rotation on each qubit to spread a bit of probability onto nearby bitstrings. Nothing is tuned or optimized.

Then it scores that circuit two different ways:

1. **As a state.** Compute its energy directly, which charges you for every amplitude that is slightly wrong.
2. **As a source of suggestions.** Keep only the bitstrings it produced, crop the Hamiltonian, and diagonalize.

> The second approach lands much closer to the exact answer. Diagonalizing throws away the circuit's amplitudes and recomputes them from scratch, so the circuit only has to visit the right bitstrings, not get their weights right.

What you will do

1. Look at a complete Hamiltonian as an 8-by-8 table, with the field strength on a slider.
2. See how the ground state's weight spreads across bitstrings, and how that changes with g.
3. Write the cropping step yourself, in about three lines. This is the piece Labs 2 and 3 build on.
4. Sample a rough circuit and compare the two ways of scoring it.
5. Sweep the circuit's rotation angle to find the balance between circuit quality and sample variety.

⏱ **About 50 minutes.** Almost none of that is waiting on cells; it is reading and thinking.

How the notebook works

Run the cells in order. You will see two markers throughout.

- 🧠 **Checkpoints** ask a question and hide the answer in a drop-down. Try to answer before you open it; that is where most of the learning happens.
- ✍️ **Your turn** cells ask you to write code between the `YOUR CODE HERE` markers. A worked solution is in a drop-down right underneath.

If you have not filled in an exercise yet, the notebook stops with a message containing a ✏️. That is the notebook reminding you, not something broken.

Three ways to run this lab

You get a single `.ipynb` file, and it is fully self-contained. There is no `requirements.txt` to fetch, no repository to clone, and no other files it depends on.

OPTION	WHERE IT RUNS	WHAT YOU INSTALL
Your own machine	Local Jupyter	Uncomment the `%pip install` line in the first cell
Google Colab	Browser, Google account	Uncomment the `%pip install` line in the first cell
qBraid Lab	Browser, free account	Build one environment, then leave that line commented

> You do not need an IBM Quantum account. Everything in this lab runs locally, so it uses no quantum processor time and no credits.

Options 1 and 2 · your machine, or Colab

On your own machine

```
python3 -m venv .venv
source .venv/bin/activate
pip install jupyterlab
jupyter lab
```

Then open the notebook and uncomment the `%pip install ...` line in its first cell, once.

- You need Python **3.11 to 3.14** on macOS or Linux.
- On Windows, use WSL, Colab, or qBraid. Parts of the stack used later in the series have no native Windows builds.

On Google Colab

Upload the `.ipynb` file, then uncomment and run that same first-cell line. Nothing else changes.

Option 3 · qBraid Lab

A JupyterLab environment that runs in your browser, with a free tier. You build the environment once and reuse it.

1. Sign in at **lab.qbraid.com** (https://lab.qbraid.com).
2. Drag the `.ipynb` file into the file browser.
3. Open the **Environments** panel (upper right), then **Add** → **Create Environment**. Name it `fall-fest-sqd` and paste in this package list:

```
numpy>=2.3.2
qiskit==2.5.1
qiskit-aer==0.17.2
qiskit-addon-sqd==0.12.1
ffsim>=0.0.81
pyscf==2.14.0
matplotlib==3.11.1
pylatexenc==2.10
```

4. When **Installing…** turns into **Add kernel**, click **Add kernel**.
5. Open the notebook, pick the `fall-fest-sqd` kernel in the upper right, and leave the `%pip install` line commented. The environment already has everything.

Getting started

Open `01_sample_then_diagonalize.ipynb` and run the first cell.

- Try each checkpoint before opening the answer. They are short.
- Work in pairs if you can. Saying a prediction out loud tends to expose the shaky parts.
- If a cell stops with a ✏️ message, it is telling you which exercise still needs filling in.

> 🧭 **If this feels like a stretch rather than a challenge,** the free Use a quantum computer today (https://quantum.cloud.ibm.com/learning/en/courses/use-a-qc-today) course on IBM Quantum Learning assumes no background and is a good place to start. This lab will still be here afterward.